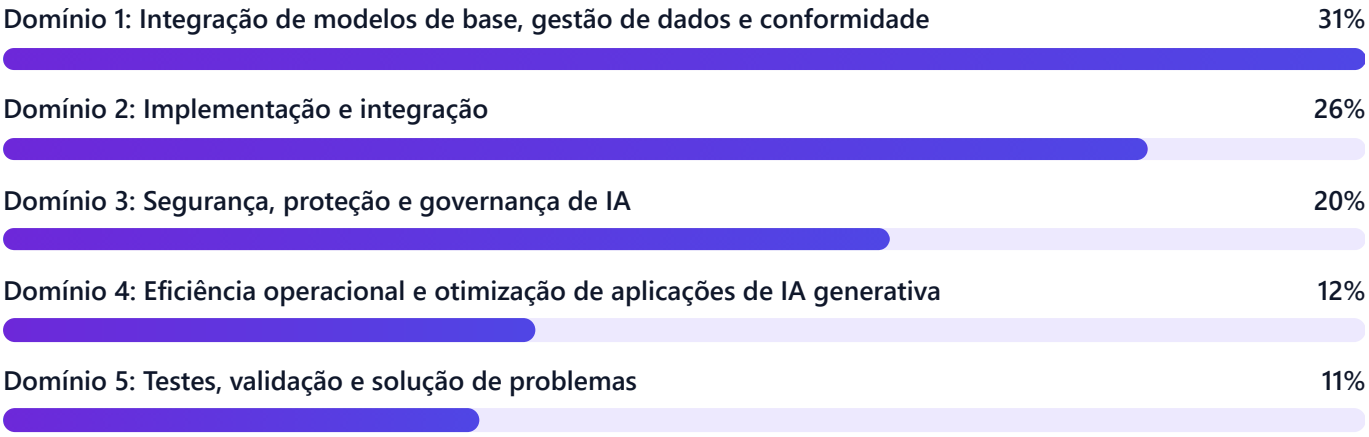
Strecht Simulados

Resumo de bolso AIP-C01

Domínios cobrados, as dicas de prova mais importantes e 10 questões comentadas para treinar.

450	6	75%
questões no curso	provas completas	nota de corte

Como é a prova AIP-C01



Material independente de preparação, sem afiliação ou endosso de Amazon Web Services (AWS). As questões são inéditas e não reproduzem a prova real.

Pronto para o exame AIP-C01?

Treine com as 450 questões em 6 provas completas: tempo do exame, explicação de cada alternativa e link da documentação para revisar.

Ver as 6 provas completas e o cupom do mês →

Dicas de prova por domínio

Domínio 3: Segurança, proteção e governança de IA

- Modelo inventando campos em chamadas de ferramenta: strict: true no toolSpec. Prompt, temperatura e guardrail reduzem, mas não garantem o schema.
- Mesmas políticas para modelos fora do Bedrock: ApplyGuardrail.
- Estereótipo e nocividade medidos antes de publicar = LLM-as-a-judge com métricas integradas sobre cenários por grupo; guardrail é proteção em produção, não avaliação.
- Chave com política editável e revogável: KMS gerenciada pelo cliente.
- Disparidade por grupo em tempo quase real: métrica personalizada com dimensões + alarme.
- Conteúdo nocivo nas duas direções + temas proibidos: um guardrail com filtros de conteúdo e tópicos negados, aplicado a entrada e saída.
- Bloquear quem insiste em jailbreak: conte as intervenções PROMPT_ATTACK do trace por usuário e aplique bloqueio temporário no autorizador.
- Avisar sem bloquear: verificação de grounding com ação NONE (modo de detecção) e pontuação lida no trace, mais citações na interface.

Domínio 2: Implementação e integração

- Texto longo em ferramenta interativa: ConverseStream (e permissão InvokeModelWithResponseStream); aumentar timeout só evita a falha.
- Prompt gigante com várias tarefas e resultados inconsistentes: encadeie prompts com saída estruturada entre as etapas.
- Pós-chamada no Amazon Connect: dispare o resumo quando o arquivo do Contact Lens chegar ao S3, não no evento de desconexão.
- Lógica própria com falha controlada no pipeline: ação de invocação do Lambda + PutJobFailureResult.
- Agente esquece ao trocar de sessão: persista o estado com um session manager (AgentCore Memory), não confie na memória da sessão.
- Agente consultando legado SOAP: Lambda adaptadora como alvo do AgentCore Gateway. O agente fala MCP e JSON; o XML fica na camada de adaptação.
- Revisão humana com espera longa no Step Functions: Choice pela confiança + .waitForTaskToken. A2I está fechado a novos clientes.
- Ver o texto na tela conforme sai: ConverseStream + Lambda response streaming + REST API em modo STREAM.

Domínio 5: Testes, validação e solução de problemas

- RAG instável para a mesma pergunta: fixe índice e embeddings e compare cada etapa em staging.

- Critério subjetivo que só especialistas da casa sabem julgar: avaliação humana do Bedrock com equipe própria e métricas customizadas.
- Chunks vazios em PDF digitalizado: problema de parsing. Use BDA ou FM como parser e sincronize de novo.
- Problema que só aparece com dados reais, sem afetar usuários: shadow testing + dataset de regressão.
- Comparar modelos: mesmo dataset, mesmo juiz, mesmas métricas, uma avaliação por tarefa; não misture tarefas numa média.
- Erro com critério exato (rota, parâmetro): validador determinístico contra o contrato, não juiz LLM.
- Bloquear regressão antes do usuário: gate de métricas no pipeline, não monitoramento em produção.
- Recuperação piorou só para conteúdo novo, sem mudar chunking nem prompt: suspeite do modelo de embeddings e prove com avaliação retrieve-only.

Domínio 1: Integração de modelos de base, gestão de dados e conformidade

- Lógica de negócio ou chamada a sistema interno entre prompts encadeados: nó Lambda no Bedrock Flows.
- Formatos misturados nos dados: normalize em código (Lambda) antes do prompt; não deixe o modelo converter.
- Gravação estéreo com um lado em cada canal: ChannelIdentification. Remover palavras: filtro de vocabulário com remove (mask troca por asteriscos).
- RAG devolvendo políticas conflitantes: tire as versões antigas da fonte (histórico no Versionamento do S3) e sincronize. Prompt e reranker não limpam o índice.
- Dados faltando no pedido: pergunte antes de chamar o FM (Choice + task token no Step Functions), em vez de deixar o modelo supor.
- Documento revogado aparecendo na resposta: filtro de metadados por data/status no Retrieve, não reranker nem instrução no prompt.
- Quem alterou um objeto do S3: eventos de dados do CloudTrail (não vêm por padrão). Conteúdo anterior: Versionamento do S3.
- Endpoints do SageMaker não servem modelos do Bedrock. Para versionar e dividir tráfego entre modelos do Bedrock, use uma camada de orquestração com configuração central.

Domínio 4: Eficiência operacional e otimização de aplicações de IA generativa

- LLM em endpoint com picos e ociosidade: inference components + target tracking por concorrência + escala a zero.
- Milhões de itens, sem pressa, menor custo e sem orquestrar chamadas: inferência em lote com S3. Flex é para chamadas síncronas tolerantes a latência.
- Latência de streaming: compare TimeToFirstToken e tokens de saída por segundo, não a média de InvocationLatency.

- Um painel com métricas, logs e custo de fontes diferentes, sem operar servidor: Amazon Managed Grafana com várias fontes de dados.
- Uso baixo com picos imprevisíveis: sob demanda; Provisioned Throughput compensa com uso alto e constante.
- Muitas perguntas reformuladas: cache semântico. Contexto estático enorme em cada prompt: RAG com só os trechos relevantes. Capacidade provisionada não reduz tokens.
- Dimensionar o Bedrock: tokens de entrada e de saída por minuto no pico. Média diária e contagem de requisições enganam.
- Detectar falha sem tráfego de usuários e validar a resposta: canário do CloudWatch Synthetics com uma conversa de teste.

10 questões para treinar

Responda antes de olhar o gabarito, no fim do material.

Questão 1 · Domínio 3: Segurança, proteção e governança de IA

Um varejista expôs um assistente de compras público que usa o Amazon Bedrock Guardrails com filtro de ataques de prompt, tópicos negados e filtro de informações sensíveis. Em um teste de intrusão, a equipe vermelha conseguiu mapear as regras porque a aplicação devolve ao usuário o nome da política que bloqueou cada tentativa. A segurança quer que o usuário receba uma resposta neutra, mas que o time continue investigando qual política foi acionada. Qual solução atende a esses requisitos?

A. Configurar o guardrail para mascarar os prompts bloqueados com o identificador da política, o que esconde o texto do atacante na resposta exibida pela aplicação
B. Retirar o filtro de ataques de prompt e confiar nos tópicos negados, que bloqueiam as tentativas de manipulação sem informar ao usuário qual tópico foi identificado na conversa
C. Trocar a ação dos filtros de bloqueio para detecção, para que o guardrail registre as violações e deixe o modelo responder normalmente ao usuário
D. Definir mensagens genéricas de bloqueio para prompts e respostas no guardrail e devolver só esse texto ao cliente, mantendo a avaliação detalhada do trace nos logs internos

Questão 2 · Domínio 5: Testes, validação e solução de problemas

Uma consultoria gera relatórios de auditoria com a Converse API do Amazon Bedrock. Vários relatórios terminam no meio de uma frase, e o campo stopReason da resposta traz max_tokens. O prompt e a pergunta estão bem abaixo do limite de contexto do modelo. Qual ajuste resolve o problema?

A. Aumentar o maxTokens em inferenceConfig ou pedir ao modelo um relatório mais conciso, dentro do limite de saída
B. Remover as stop sequences configuradas na chamada, que estão interrompendo a geração antes do fim do relatório
C. Reduzir o tamanho da pergunta enviada pelo usuário, liberando espaço para a resposta dentro da janela de contexto
D. Trocar a Converse pela ConverseStream, que entrega a resposta em partes e não tem o mesmo limite de saída

Questão 3 · Domínio 5: Testes, validação e solução de problemas

Um jornal resume notas de agências de notícias com um modelo do Amazon Bedrock. O template junta, num único bloco de texto corrido, as instruções de estilo, uma nota de exemplo com o resumo esperado e a nota do dia. Em parte das respostas, o resumo mistura fatos da nota de exemplo com os da nota real ou repete frases das instruções. Os resumos não estão sendo cortados, e a equipe não pode trocar de modelo. Qual ajuste ataca a causa?

A. Indexar as notas de agência numa base de conhecimento e recuperar a nota do dia com RetrieveAndGenerate, em vez de enviá-la junto do exemplo e das instruções no mesmo bloco de texto do prompt enviado ao modelo
B. Remover a nota de exemplo e aumentar a temperatura, para que o modelo produza resumos mais variados e menos presos ao texto do template usado pela redação
C. Separar instruções, exemplo e nota do dia em seções nomeadas e delimitadas, levar as instruções ao prompt de sistema, apontar a seção a resumir e proibir que o exemplo apareça na resposta
D. Aumentar o maxTokens da resposta, porque o resumo está sendo truncado e o modelo completa o texto com trechos do exemplo e das instruções de estilo

Questão 4 · Domínio 3: Segurança, proteção e governança de IA

Uma seguradora resume laudos de vistoria com um modelo do Amazon Bedrock e passa cada resumo pela ApplyGuardrail com a verificação de grounding contextual, usando o laudo como fonte. O comitê de IA responsável quer quantificar a incerteza dos resumos por tipo de sinistro, ser alertado quando a confiança média cair e enviar à revisão humana os resumos menos confiáveis, sem bloquear a maioria deles. Qual solução atende a esses requisitos?

A. Elevar o limiar de grounding do guardrail para 0,99, para que os resumos incertos sejam bloqueados na hora e reescritos pelos analistas de sinistro da seguradora
B. Usar a métrica InvocationsIntervened do namespace AWS/Bedrock/Guardrails como medida de confiança dos resumos de vistoria e revisar no fim de cada dia os resumos que foram bloqueados pelo guardrail
C. Pedir ao modelo que informe no fim de cada resumo uma nota de 0 a 10 para a própria confiança e mandar à revisão humana os resumos com nota abaixo de 7
D. Publicar como métricas do CloudWatch, com a dimensão tipo de sinistro, as pontuações de grounding e relevância do guardrail, alarmar sobre a média e revisar os resumos abaixo de um limiar

Questão 5 · Domínio 3: Segurança, proteção e governança de IA

Um banco usa um modelo do Amazon Bedrock para recomendar a gerentes o limite de crédito de pequenas empresas, com base em documentos recuperados de uma base de conhecimento. A diretoria de IA responsável quer que o gerente veja por que a recomendação foi feita e quais documentos a sustentam, mas a segurança proíbe expor o prompt de sistema, que contém regras internas da política de risco. A interface precisa exibir esses elementos em um formato fixo. Qual solução atende a esses requisitos?

A. Exigir saída estruturada com JSON Schema no outputConfig.textFormat da Converse, com campos de recomendação, resumo da justificativa e evidências com ID do documento e trecho, mantendo o prompt de sistema no backend
B. Pedir no prompt que o modelo explique a recomendação em texto livre ao final da resposta, sem revelar as regras internas, e exibir esse parágrafo na interface
C. Ativar o raciocínio estendido do modelo e exibir ao gerente o conteúdo de raciocínio completo devolvido pela API, que já traz a justificativa passo a passo da recomendação de crédito com as regras aplicadas em cada etapa da análise
D. Mostrar ao gerente o prompt de sistema e o contexto recuperado ao lado da resposta, já que a transparência sobre as instruções facilita explicar a recomendação

Questão 6 · Domínio 3: Segurança, proteção e governança de IA

Uma cooperativa de crédito ajustou um modelo de linguagem no Amazon SageMaker AI para analisar pedidos de renegociação. O regulador pede documentação estruturada e versionada do modelo, com uso pretendido, classificação de risco, dados de treinamento, métricas de avaliação e limitações conhecidas, e a possibilidade de exportar o documento para auditoria. Qual recurso atende a esse pedido com o MENOR esforço?

A. Linhagem das tabelas de treinamento registrada no AWS Glue Data Catalog do projeto
B. Metadados de versão e status de aprovação do pacote do modelo no SageMaker Model Registry
C. Model cards do Amazon SageMaker AI, preenchidos e versionados para o modelo ajustado
D. AWS AI Service Cards publicados pela AWS para os serviços de IA usados no projeto

Questão 7 · Domínio 3: Segurança, proteção e governança de IA

Um marketplace de artesanato permite que vendedores enviem fotos dos produtos para um modelo multimodal do Amazon Bedrock gerar a descrição do anúncio. Algumas fotos enviadas têm conteúdo sexual ou violento, e a política exige bloquear essas imagens antes que cheguem ao modelo, com os mesmos níveis de filtro já usados para o texto. A equipe quer reaproveitar o guardrail que já protege os textos. Qual configuração atende a esses requisitos?

A. Aplicar a verificação de grounding contextual do guardrail às descrições geradas, bloqueando as respostas que não se apoiem no conteúdo da foto enviada pelo vendedor
B. Adicionar ao guardrail tópicos negados que descrevam conteúdo sexual e violento, que passam a ser avaliados também sobre as imagens enviadas junto com o texto
C. Habilitar a modalidade de imagem nos filtros de conteúdo Sexual e Violence do guardrail, enviar as fotos em PNG ou JPEG e aplicar o guardrail na chamada ao modelo
D. Ativar o filtro de ataques de prompt com modalidade de imagem, que classifica fotos com conteúdo sexual ou violento como tentativa de jailbreak e bloqueia o envio ao modelo

Questão 8 · Domínio 3: Segurança, proteção e governança de IA

Um hospital digital tem várias aplicações que chamam modelos do Amazon Bedrock com textos em português. A equipe de segurança quer uma política centralizada que detecte e mascare dados como CPF, número de cartão e dados de contato tanto nos prompts quanto nas respostas, que seja aplicada obrigatoriamente em cada chamada de inferência e que deixe registro das intervenções para auditoria. Qual abordagem atende a esses requisitos?

A. Configurar o Amazon Macie para analisar os prompts e as respostas em trânsito entre as aplicações e o Amazon Bedrock, mascarando os dados pessoais detectados pelo serviço antes da entrega
B. Criar um guardrail com filtro de informações sensíveis em modo de mascaramento, com tipos de PII predefinidos e regex para o CPF, e exigi-lo pela condição bedrock:GuardrailIdentifier
C. Incluir em cada aplicação uma função Lambda que chama o DetectPiiEntities do Amazon Comprehend e remove os dados pessoais do prompt antes de enviá-lo ao modelo de IA
D. Ativar o Amazon GuardDuty na conta para inspecionar as chamadas feitas ao bedrock-runtime e bloquear as requisições que contenham dados pessoais de pacientes

Questão 9 · Domínio 4: Eficiência operacional e otimização de aplicações de IA generativa

Uma empresa está otimizando os custos de uma aplicação de geração de código com um modelo de alta capacidade no Amazon Bedrock. As chamadas usam streaming, mas o time percebeu que muitos tokens gerados depois de certo ponto não são usados: os desenvolvedores interrompem a leitura ou ignoram o restante da resposta. O objetivo é reduzir custo e latência sem degradar a qualidade percebida. Qual abordagem é a MAIS adequada?

A. Ajustar o maxTokens por tipo de requisição com base nas métricas reais de consumo dos usuários e revisar o limite periodicamente
B. Definir stop sequences genéricas, iguais para cada tipo de requisição, para interromper a geração em marcadores comuns de fim de bloco
C. Migrar para um modelo menor e mais barato da mesma família, mantendo os mesmos parâmetros de geração e o mesmo prompt de sistema
D. Ativar o prompt caching no prefixo fixo do system prompt e dos exemplos, reaproveitando os tokens de entrada entre requisições

Questão 10 · Domínio 1: Integração de modelos de base, gestão de dados e conformidade

Uma software house oferece geração de código com o Amazon Bedrock. O front-end envia só um identificador de prompt e alguns parâmetros de negócio; o backend precisa obter o template aprovado, preencher as variáveis, chamar o modelo e registrar, para auditoria, qual versão do prompt gerou cada resposta. Qual implementação atende a esses requisitos com MENOS código próprio de gestão de templates?

A. Deixar o front-end montar o prompt completo a partir de um catálogo de templates e enviar ao backend o texto final junto com o identificador do template para rastreio
B. Guardar os templates em variáveis de ambiente da Lambda, preencher as variáveis no código da função e registrar cada chamada ao modelo no CloudWatch Logs
C. Publicar versões aprovadas dos templates no Prompt Management e, na Lambda, chamar a Converse com o ARN da versão e promptVariables, gravando o histórico no DynamoDB
D. Guardar os templates como configuração no AWS AppConfig, preencher as variáveis no código da Lambda e registrar no histórico só o nome do template usado

Gabarito comentado

1. D) Definir mensagens genéricas de bloqueio para prompts e respostas no guardrail e devolver só esse texto ao cliente, mantendo a avaliação detalhada do trace nos logs internos
Não revelar ao atacante qual regra bloqueou: mensagem genérica de bloqueio no guardrail ao usuário e trace detalhado só nos logs internos.

2. A) Aumentar o maxTokens em inferenceConfig ou pedir ao modelo um relatório mais conciso, dentro do limite de saída
Resposta cortada com stopReason max_tokens: aumente o maxTokens (ou peça concisão). stop_sequence seria outra causa.

3. C) Separar instruções, exemplo e nota do dia em seções nomeadas e delimitadas, levar as instruções ao prompt de sistema, apontar a seção a resumir e proibir que o exemplo apareça na resposta
Resposta mistura exemplo, instruções e conteúdo: delimite e nomeie as seções do prompt. Não é caso de temperatura, maxTokens nem RAG.

4. D) Publicar como métricas do CloudWatch, com a dimensão tipo de sinistro, as pontuações de grounding e relevância do guardrail, alarmar sobre a média e revisar os resumos abaixo de um limiar
Quantificar incerteza e acionar revisão: publique a pontuação de confiança como métrica com dimensões e roteie a faixa baixa para humanos.

5. A) Exigir saída estruturada com JSON Schema no outputConfig.textFormat da Converse, com campos de recomendação, resumo da justificativa e evidências com ID do documento e trecho, mantendo o prompt de sistema no backend
Explicar a decisão sem vazar instruções: saída estruturada com justificativa resumida e evidências; nunca exiba prompt de sistema ou raciocínio bruto.

6. C) Model cards do Amazon SageMaker AI, preenchidos e versionados para o modelo ajustado
Documentar um modelo próprio para regulador: model card do SageMaker AI. Service Card é da AWS; Model Registry é versão e aprovação.

7. C) Habilitar a modalidade de imagem nos filtros de conteúdo Sexual e Violence do guardrail, enviar as fotos em PNG ou JPEG e aplicar o guardrail na chamada ao modelo
Imagem nociva em app multimodal = filtros de conteúdo com modalidade IMAGE no mesmo guardrail; tópicos negados e grounding não analisam a foto.

8. B) Criar um guardrail com filtro de informações sensíveis em modo de mascaramento, com tipos de PII predefinidos e regex para o CPF, e exigi-lo pela condição bedrock:GuardrailIdentifier
PII em prompts e respostas, inclusive em português: filtro de informações sensíveis do Guardrails; CPF por regex; uso obrigatório via IAM.

9. A) Ajustar o maxTokens por tipo de requisição com base nas métricas reais de consumo dos usuários e revisar o limite periodicamente
Tokens de saída desperdiçados: maxTokens calibrado pelo uso real; prompt caching só barateia a entrada.

10. C) Publicar versões aprovadas dos templates no Prompt Management e, na Lambda, chamar a Converse com o ARN da versão e promptVariables, gravando o histórico no DynamoDB
Prompt versionado e rastreável sem renderizar no código: Prompt Management + Converse com o ARN da versão.

Pronto para o exame AIP-C01?

Treine com as 450 questões em 6 provas completas: tempo do exame, explicação de cada alternativa e link da documentação para revisar.

Ver as 6 provas completas e o cupom do mês →